

Test Case Document
for
Restaurant Management System

New test cases

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
UT_21	Table Booking Management	Verify a booking cannot be cancelled less than 30 minutes before the reserved time (REQ-3, negative)	A confirmed booking exists for a slot less than 30 minutes away	1: Submit a cancellation request for the booking 2: Query the booking status and table status	Booking reserved for 19:00; cancellation submitted at 18:45	Cancellation request is rejected with a message that a booking can only be cancelled up to 30 minutes before the reserved time; booking status remains 'Confirmed' and the table stays reserved	Not Executed	Not Executed
ST_09	Table Booking Management	End-to-end: customer cancels a booking more than 30 minutes before the reserved time; staff finds it in the dashboard, sees the table released and manually creates a new booking for that slot (REQ-3, REQ-4)	Customer has a confirmed booking for a slot more than 30 minutes away; Staff account exists	1: Customer logs in and cancels the booking 2: Log in as Staff and search for the booking in the admin dashboard 3: Check the status shown for the booking 4: Manually create a new booking for the same table and slot on behalf of a customer 5: Check that the new booking is listed in the dashboard	Existing booking ID: BK-1002 (Table 5, 2026-10-25, 19:00), cancelled at 17:00; Staff credentials; New booking: Table 5, 2026-10-25, 19:00, Party size: 2	Customer sees BK-1002 as 'Cancelled'; staff search returns BK-1002 with status 'Cancelled'; Table 5 can be booked again for 19:00; the manual booking is created and listed in the dashboard	Not Executed	Not Executed
ST_10	Non-Functional Requirements (Performance)	Performance: system under 50 concurrent users meets the response-time and error-rate limits (PERF-1, PERF-3)	System is deployed on the test server with PostgreSQL; menu items, tables and bookings are seeded; a load-testing tool is available	1: Start 50 virtual users at the same time 2: Each user repeatedly browses the menu, checks table availability and places an order for the whole test period 3: Record the response time of every page load and REST API request 4: Count the HTTP 5xx responses 5: Calculate the percentage of responses within 2 seconds and the percentage of HTTP 5xx responses	Concurrent users: 50; Test period: 5 minutes; Operations: menu browse, availability check, order placement	At least 95% of page loads and REST API responses complete within 2 seconds; HTTP 5xx error rate is no more than 1%; all 50 users are served for the full test period	Not Executed	Not Executed
SV_01	Security Validation	Verify passwords are stored only as salted PBKDF2-SHA256 or Argon2 hashes and do not appear in application logs (SEC-1)	Application logging is enabled; the emails used are not already registered	1: Register two new users with the same password 2: Inspect the stored password value of both users in the database 3: Compare the two stored values 4: Search the application log for the plain-text password	customer4@example.com and customer5@example.com, Password: Cust@7777 (for both)	Both stored values are in PBKDF2-SHA256 or Argon2 hash format with a salt; the two stored values are different; the plain-text password is not present in the database or in the application log	Not Executed	Not Executed
SV_02	Security Validation	Verify all pages, REST endpoints and WebSocket connections use TLS 1.2 or higher and HTTP requests are redirected to HTTPS (SEC-2)	System is deployed with HTTPS enabled; an order exists for the order-tracking page	1: Open the site URL using http:// 2: Call a REST endpoint using http:// 3: Check the TLS version used by the https connection 4: Try to connect to the server using TLS 1.1 5: Open the order-tracking page and check the WebSocket URL	http://<server-host>/ and http://<server-host>/api/menu; Order ID: OR-2001	Both http requests are redirected to https; the connection uses TLS 1.2 or higher; the TLS 1.1 connection is refused; the WebSocket connection uses wss:// and connects successfully	Not Executed	Not Executed

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
SV_03	Security Validation	Verify role-based access control on API endpoints: 401 for unauthenticated requests and 403 for unauthorised roles (SEC-3, REQ-13)	Customer, Staff and Admin accounts exist; booking BK-1003 owned by another customer and order OR-2001 exist	1: Without logging in, call GET /api/bookings and POST /api/orders 2: Log in as Customer and call POST /api/menu and PATCH /api/orders/OR-2001/status 3: As the same Customer, cancel BK-1003 using PATCH /api/bookings/BK-1003/cancel 4: Log in as Staff and call POST /api/menu 5: Log in as Admin and call POST /api/menu	Customer, Staff and Admin credentials; Booking ID: BK-1003; Order ID: OR-2001; Menu item: 'Veg Soup', Price: 150, Category: 'Starters'	Step 1 returns 401 Unauthorized; steps 2, 3 and 4 return 403 Forbidden and no data is changed; step 5 succeeds	Not Executed	Not Executed
SV_04	Security Validation	Verify SQL injection input is treated as plain data and is not executed (SEC-4)	Menu items and bookings exist in the database	1: Submit the login form with SQL text in the email field 2: Call GET /api/menu with SQL text in the category filter 3: Query the database for the menu and bookings tables and their record counts	Email: admin' OR '1'='1, Password: abc; Category: Starters'; DROP TABLE menu_item;--	Login is rejected; the menu request returns an empty result or a 400 validation error, not a 500 error and no unrelated records; both tables still exist with the same record counts	Not Executed	Not Executed
SV_05	Security Validation	Verify cross-site-scripting input is escaped on output and is not executed (SEC-4)	A Staff account exists; table 6 is free for the slot	1: Register a new customer with a script tag as the name 2: Log in as this customer and book a table 3: Log in as Staff and open the Bookings section of the dashboard 4: View the page source and check the stored value	Name: <script>alert('XSS')</script>, Email: customer6@example.com, Password: Cust@8888; Booking: Table 6, 2026-10-25, 20:00, Party size: 2	No alert is shown; the customer name appears as plain text in the dashboard; the page source shows the escaped text (<script>); the booking is created normally	Not Executed	Not Executed
SV_06	Security Validation	Verify state-changing requests without a valid CSRF token are rejected (SEC-5)	Customer is logged in and has a valid session cookie	1: Send POST /api/bookings with the session cookie and no CSRF token 2: Send the same request with an invalid CSRF token 3: Send PATCH /api/bookings/BK-1004/cancel with no CSRF token 4: Send the POST request again with a valid CSRF token 5: Query the database for new bookings	POST /api/bookings {table:5, date:'2026-10-26', time:'19:00', guests:2}; Invalid token: 'abc123invalid'; Booking ID: BK-1004	Steps 1, 2 and 3 return 403 Forbidden and no booking is created or cancelled; step 4 returns 201 Created and one new booking exists	Not Executed	Not Executed
SV_07	Security Validation	Verify an account is locked for 15 minutes after 5 consecutive failed login attempts (SEC-6)	A registered account exists; the account has no earlier failed attempts	1: Log in with a wrong password 5 times in a row 2: Log in with the correct password immediately after 3: Wait 15 minutes 4: Log in with the correct password	Email: customer7@example.com, Correct password: Cust@2468, Wrong password: Wrong@111	The 5 wrong attempts are rejected; the login in step 2 is rejected because the account is locked; the login in step 4 succeeds	Not Executed	Not Executed
SV_08	Security Validation	Verify an audit-log entry is written for every menu change, staff booking override and order-status change, and entries are retained for at least 90 days (SEC-7)	Admin and Staff accounts exist; audit log can be viewed; booking BK-1003 and order OR-2002 (status Received) exist; an audit entry	1: Log in as Admin and change the price of a menu item 2: Log in as Staff and manually cancel a booking from the dashboard 3: As Staff, change the status of an order to 'Preparing'	Item: 'Veg Soup', price 150 - > 160; Booking ID: BK-1003; Order ID: OR-2002	The audit log has one entry for each of the three actions with the user ID, action and timestamp; the retention setting is at least 90 days and the 90-day-old entry is still present	Not Executed	Not Executed

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
			dated 90 days earlier exists	4: Inspect the audit log for these three actions 5: Check the audit-log retention setting and the entry dated 90 days earlier				
SV_09	Security Validation	Verify the session cookie has the HttpOnly and Secure flags set (SEC-8)	System is deployed with HTTPS enabled; a registered account exists	1: Log in over https 2: Inspect the session cookie in the browser developer tools 3: Run document.cookie in the browser console	Email: customer2@example.com, Password: Cust@5678	The session cookie shows the HttpOnly and Secure flags; the session cookie is not listed in the document.cookie output	Not Executed	Not Executed
SV_10	Security Validation	Verify the session is invalidated after 30 minutes of inactivity on the deployed system (SEC-8, REQ-14)	System is deployed; a registered account exists	1: Log in and note the session cookie 2: Leave the session idle for 31 minutes 3: Open a protected page using the same session cookie 4: Log in again	Email: customer2@example.com, Password: Cust@5678; Inactivity period: 30 minutes (idle for 31 minutes)	The request in step 3 returns 401 Unauthorized, the session is no longer valid and the user is asked to log in again; the login in step 4 succeeds	Not Executed	Not Executed

Modified test cases

Test Case ID	Name of Module	Test case description	Pre-conditions	Test Steps	Test data	Expected Results	Actual Result	Test Result
IT_07	Food Ordering and Order Tracking	Verify the customer's order-tracking screen updates in real time upon receiving WebSocket status pushes, with at least 95% of updates within 1 second (REQ-10, PERF-2)	Customer's order-tracking page is open with an active WebSocket connection for each of 20 orders with status 'Received'	1: Trigger an order-status update on the backend for each of the 20 orders 2: Record the time each status change is saved and the time the tracking screen shows the new status, without refreshing 3: Calculate the delay for each update 4: Count the updates with a delay of 1 second or less	Order IDs: OR-2001 to OR-2020, status change: 'Received' -> 'Preparing'	Each order-tracking screen updates to show 'Preparing' with no manual refresh; at least 19 of the 20 updates (95%) appear within 1 second of the status change being saved	Not Executed	Not Executed
UT_20	User Authentication and Role Management	Verify a logged-in user's session/token is invalidated after 30 minutes of inactivity (REQ-14, SEC-8)	User is logged in and holds a valid session/token	1: Log in and note the issued session/token 2: Remain inactive for 31 minutes 3: Attempt to access a protected page using the same token	Inactivity period: 30 minutes (idle for 31 minutes)	The session/token is rejected as expired/invalid (401 Unauthorized) and the user is prompted to log in again	Not Executed	Not Executed